

과정 1 · 영상 4

코딩 에이전트로 KODEX 200 가격을 불러와 노트북에 표시한다

첫 실습

데이터를 파일로 저장하고, 새 커널에서 처음부터 실행해 결과를 확인하는 첫 실습

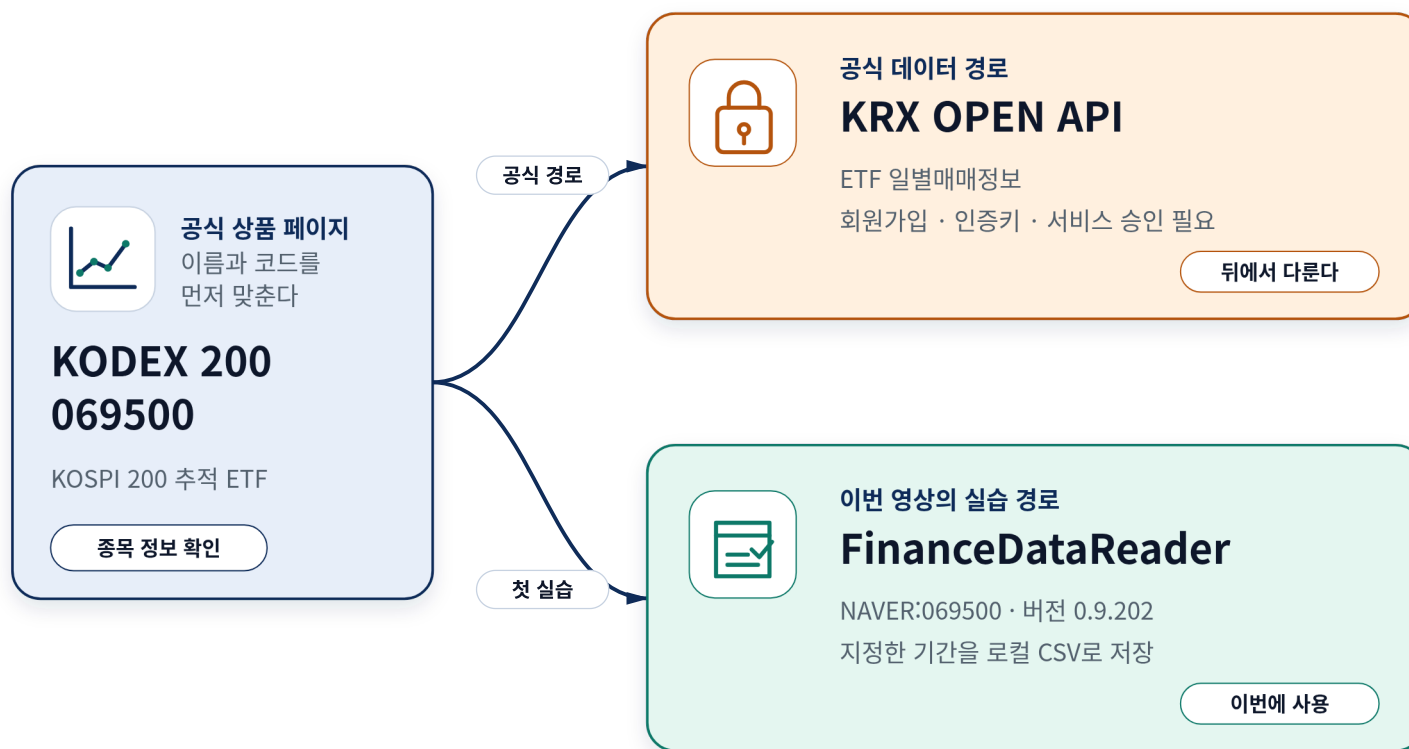
에이전트에게 작업을 요청하고 결과까지 직접 확인한다

KODEX 200 CSV와 노트북을 만든 뒤 새 커널에서 처음부터 다시 실행합니다.



어떤 종목을 어디에서 불러올지 먼저 정한다

KODEX 200은 069500이고, 이번 실습에서는 NAVER 경로로 가격을 불러옵니다.



코딩 에이전트에게 작업 범위와 확인 방법을 함께 알려 준다

종목과 기간, 작업 폴더, 만들 파일, 화면에 표시할 결과, 마무리 안내를 요청문에 적습니다.



준비한 프롬프트를 에이전트에게 전달한다

● 실제 데모 **확인 방법** 표·그래프·Run All · 만든 뒤 직접 확인할 항목까지 함께 전달

for name in os.listdir('course-01-practice'): os.system('cd ' + name + ' && python3 main.py')

› 현재 연 course-01-practice 폴더 안에서만 작업해 주세요.

KODEX 200(069500)의 2024-01-02부터 2024-03-29까지 일별 가격을 확인하는 실습 자료를 만들어 주세요. 데이터 공급자는 FinanceDataReader의 NAVER:069500 경로를 사용합니다.

다음 파일을 만들어 주세요.

- data/kodex200_price.csv
- notebooks/kodex200_price.ipynb
- requirements.txt

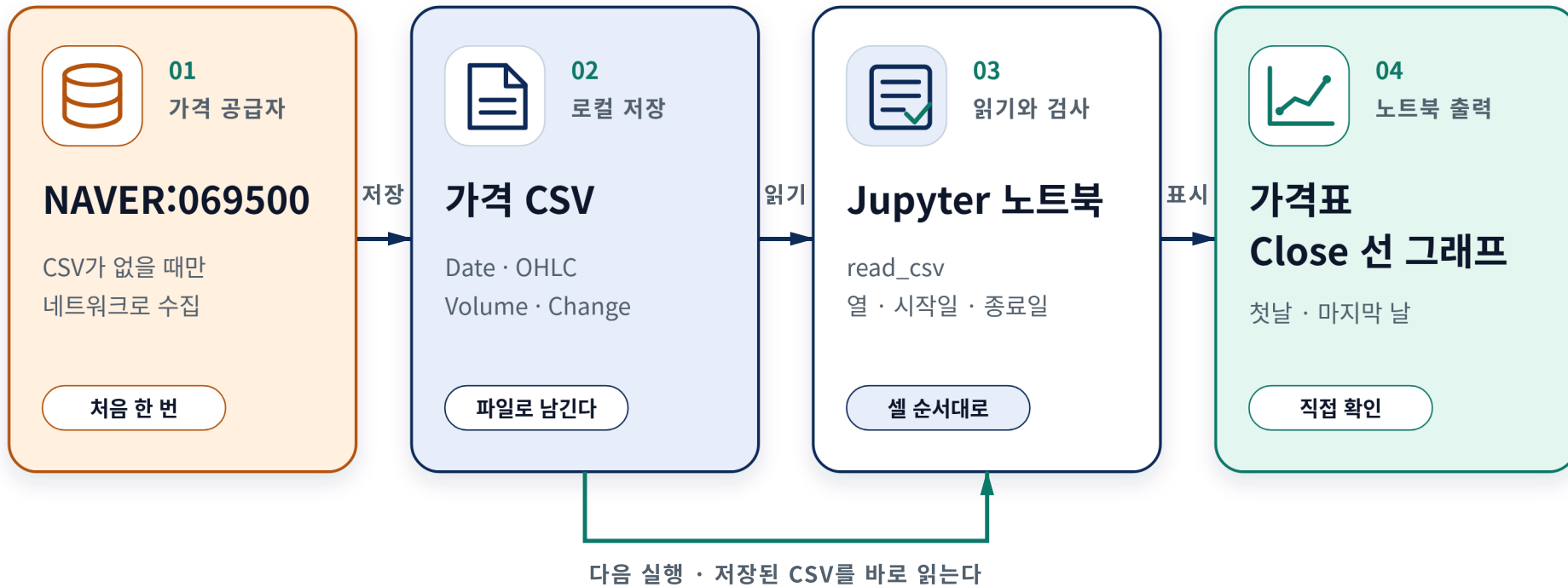
노트북은 CSV가 없을 때만 데이터를 받아 저장하고, 이후에는 저장된 CSV를 다시 읽어야 합니다. 날짜가 포함된 가격표의 처음과 마지막 일부를 보여 주고, Date를 가로축, Close를 세로축으로 한 선 그래프를 표시해 주세요. 빈 데이터, 필수 열 누락, 시작일과 종료일 불일치는 명확한 오류로 알려 주세요. 커널을 다시 시작한 뒤 Run All로 처음부터 실행할 수 있어야 합니다.

작업을 마치면 만든 파일, 실행 방법, 사용한 라이브러리와 데이터 공급자, 확인이 필요한 한계를 요약해 주세요. 매매 규칙이나 백테스트는 만들지 마세요.

■

CSV에 저장한 가격을 노트북에서 표와 그래프로 확인한다

가격은 CSV에 보관하고, 노트북은 그 파일을 다시 읽어 결과를 표시합니다.



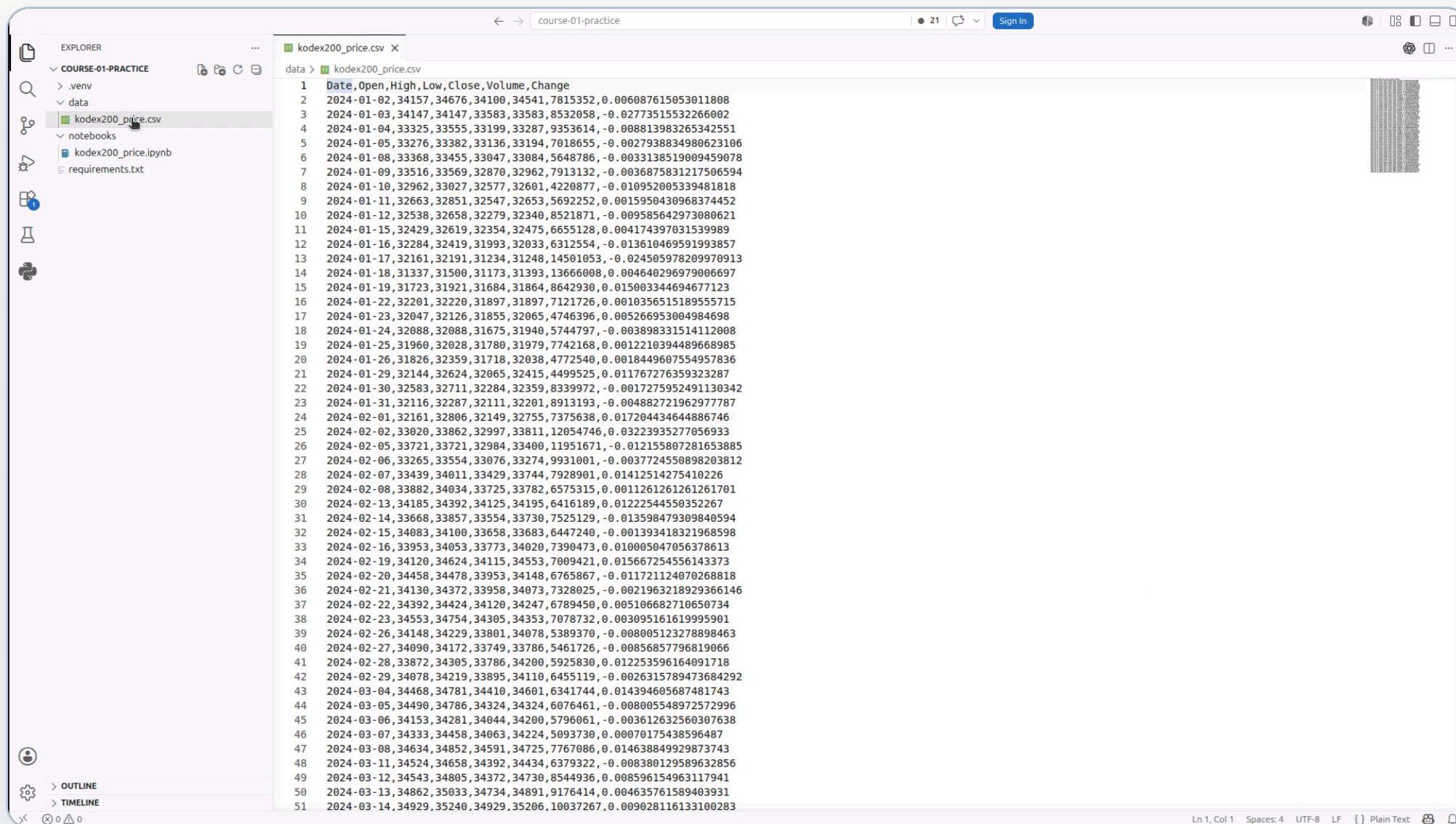
01

실제 파일을 열어 결과를 확인한다

CSV와 노트북을 화면에서 직접 확인합니다

CSV를 열어 날짜와 가격이 저장됐는지 본다

● 실제 데모 날짜와 값 실제 행이 저장됨 · 2024-01-02부터 2024-03-29



가격 데이터의 일곱 개 열을 먼저 읽어 본다

Date는 거래일이고, Open·High·Low·Close는 하루의 가격, Volume과 Change는 거래량과 등락률입니다.

Date

거래일

가격이 기록된 날짜



하루의 가격

Open

시가

거래가 시작될 때의 가격

High

고가

그날 가장 높았던 가격

Low

저가

그날 가장 낮았던 가격

Close

종가

거래가 끝날 때의 가격

Volume

거래량

그날 거래된 수량



Change

등락률

전일 종가와 비교한 변화율



노트북은 CSV가 있을 때 그 파일을 다시 읽는다

첫 수집과 이후 실행을 나누면 같은 파일로 표와 그래프를 다시 만들 수 있습니다.

• • • kodex200_price.ipynb

```
# CSV가 없을 때만 가격을 내려받아 저장합니다.
if not csv_path.exists():
    downloaded = fdr.DataReader(SYMBOL, START, END)
    downloaded.to_csv(csv_path)
```

```
# 이후 작업은 저장해 둔 CSV를 다시 읽습니다.
prices = pd.read_csv(csv_path, parse_dates=["Date"])
```

```
# 표와 그래프는 같은 prices에서 만듭니다.
display(prices.head())
ax.plot(prices["Date"], prices["Close"])
```

01 CSV 확인

파일이 없을 때만 수집

02 CSV 읽기

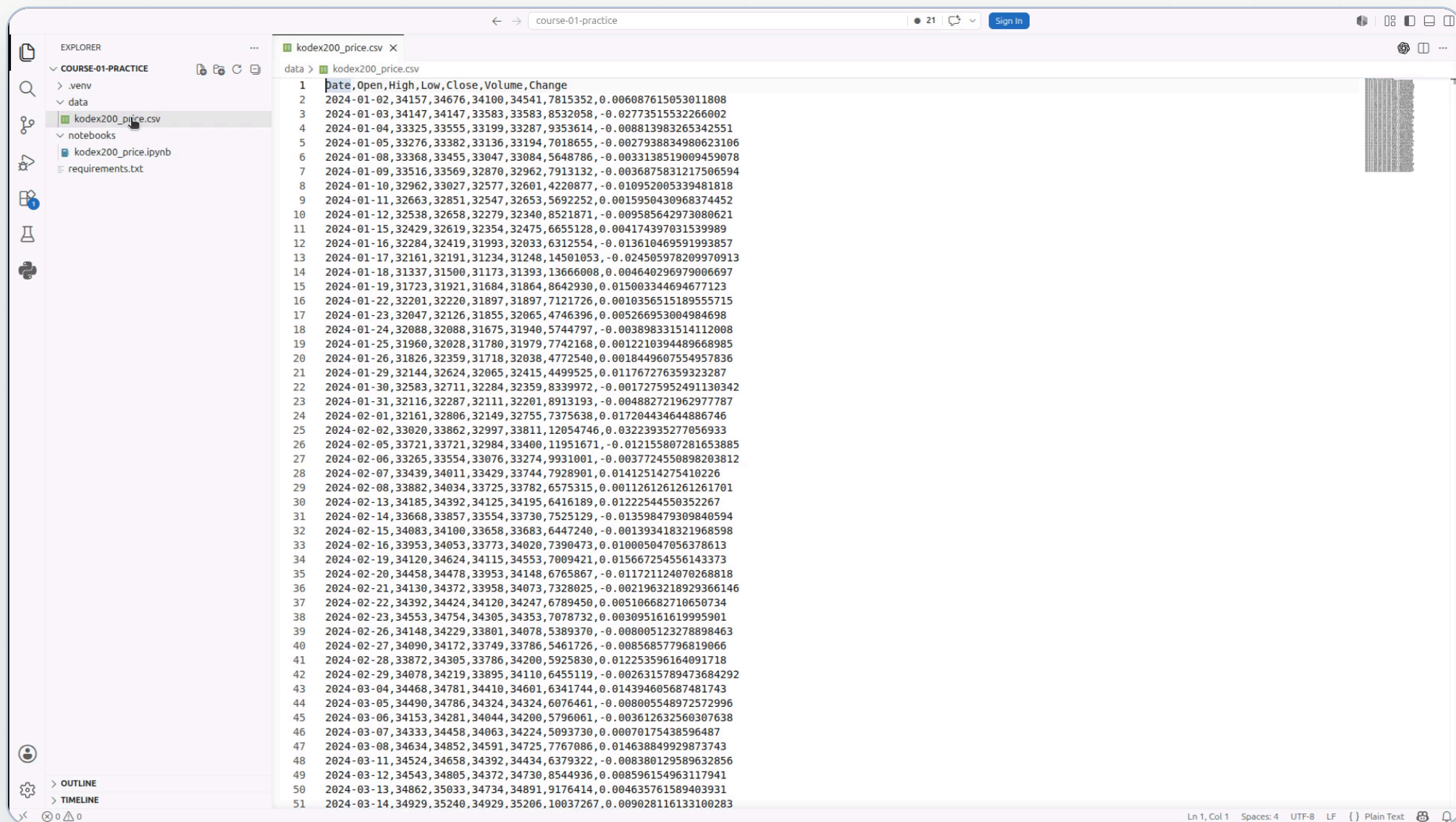
이후 실행은 로컬 파일 사용

03 결과 표시

표와 그래프가 같은 데이터 사용

실행 결과는 코드 바로 아래에서 확인한다

- 실제 데모 **입력 파일** 같은 로컬 CSV · 화면 값과 파일 값을 맞춰 봄



이번 실습에서 불러온 가격 데이터는 모두 61행이다

행 수와 함께 날짜 범위, 첫날과 마지막 날의 종가도 확인합니다.

가격 데이터

61행

2024-01-02부터 2024-03-29

첫날 종가

34,541

2024-01-02

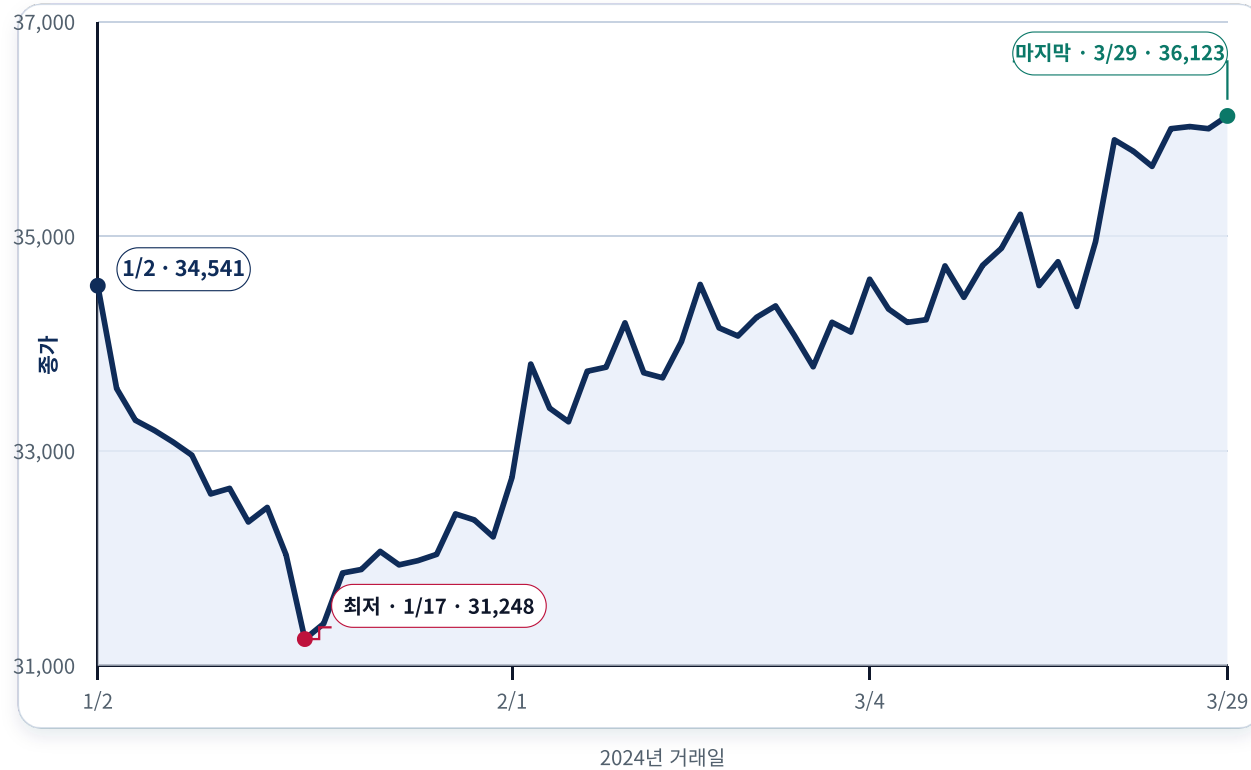
마지막 날 종가

36,123

2024-03-29

Close 열을 날짜 순서로 연결해 종가 흐름을 확인한다

표에서 읽은 종가를 선으로 이으면 날짜 순서와 값의 흐름을 빠르게 살펴볼 수 있습니다.



02

새 커널에서 처음부터 다시 실행한다

이전 메모리를 지우고 저장된 파일만으로 다시 확인합니다

새 커널에서 실행하면 앞서 실행한 셀에 기대는 문제를 찾을 수 있다

일부 셀만 실행했을 때와 첫 셀부터 끝까지 실행했을 때를 구분합니다.



Restart를 눌러 이전 실행 상태를 지운다

- 실제 데모 다음 단계 Run All · 첫 셀부터 다시 실행

The screenshot shows a Jupyter Notebook titled 'course-01-practice'. The left sidebar shows a file explorer with the following structure:

- COURSE-01-PRACTICE
 - .env
 - data
 - kodex200_price.csv
 - notebooks
 - kodex200_price.ipynb (selected)
 - requirements.txt

The main area displays the notebook content:

KODEX 200 가격 확인

FinanceDataReader에서 받은 가격을 CSV로 저장한 뒤, 같은 파일을 다시 읽어 표와 그래프로 확인합니다.

```
[1] # 파일 경로, 데이터 수집, 표 처리, 그래프에 필요한 도구를 불러옵니다.
from pathlib import Path

import FinanceDataReader as fdr
import matplotlib.pyplot as plt
import pandas as pd

# 실제로 사용할 종목과 조회 기간을 한곳에 모아 둡니다.
START = "2024-01-02"
END = "2024-03-29"
SYMBOL = "NAVER:069500"
```

```
[2] # 노트북 폴더에서 실행해도 프로젝트의 data 폴더를 찾도록 기준 경로를 정합니다.
cwd = Path.cwd()
project_root = cwd.parent if cwd.name == "notebooks" else cwd
csv_path = project_root / "data" / "kodex200_price.csv"
csv_path.parent.mkdir(parents=True, exist_ok=True)
print(f"저장 위치: {csv_path.relative_to(project_root)}")
```

PosixPath('/tmp/aimlquant-video4-demo/course-01-practice/data/kodex200_price.csv')

```
[3] # CSV가 없을 때만 가격을 내려받아 저장합니다.
if not csv_path.exists():
    downloaded = fdr.DataReader(SYMBOL, START, END)
    if downloaded.empty:
        raise RuntimeError("가격 데이터가 비어 있습니다.")
    downloaded.index.name = "Date"
    downloaded.to_csv(csv_path)

print(f"사용할 CSV: {csv_path.relative_to(project_root)}")
```

사용할 CSV: /tmp/aimlquant-video4-demo/course-01-practice/data/kodex200_price.csv

At the bottom right, a status bar indicates: **Restarting Kernel** : .env (3.12.3) (Python 3.12.3): Waiting for Jupyter...

Run All로 첫 셀부터 마지막 셀까지 실행한다

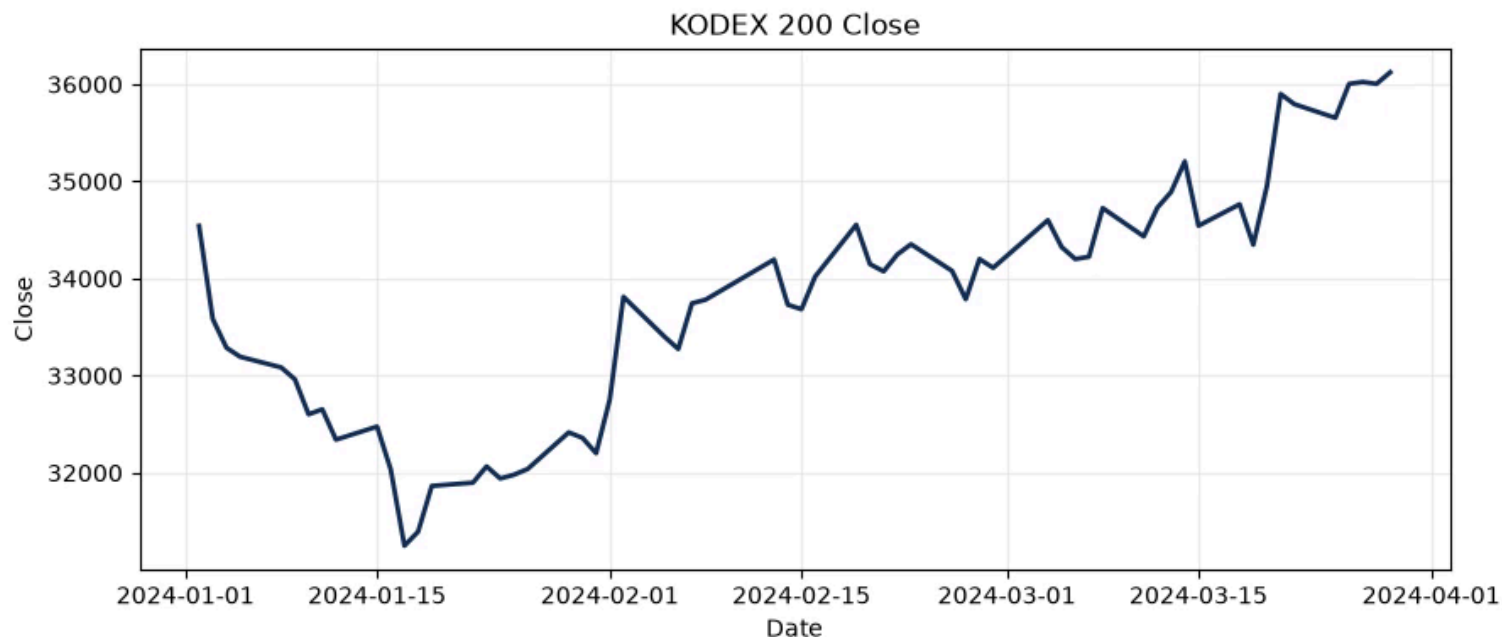
• 실제 데모 결과 표와 그래프 재현 · 대기 구간은 2배속 편집

59	2024-03-28	36009	36191	35914	36004	4985227	-0.000555
60	2024-03-29	36162	36266	35949	36123	5267314	0.003305

```
# Date를 가로축, Close를 세로축으로 종가 흐름을 그림니다.
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(prices["Date"], prices["Close"], color="#0F2C59", linewidth=2)
ax.set(title="KODEX 200 Close", xlabel="Date", ylabel="Close")
ax.grid(alpha=0.2)
plt.show()
```

[6] ✓ 0.0s

...



처음부터 다시 실행한 뒤 커널·셀·표·그래프를 확인한다

커널 선택과 실행 상태를 보고, 표와 그래프가 다시 나타났는지 확인합니다.

01



커널

.venv가 선택돼 있다

02



실행 상태

모든 셀이 오류 없이 끝났다

03



표

시작일과 종료일이 다시 보인다

04

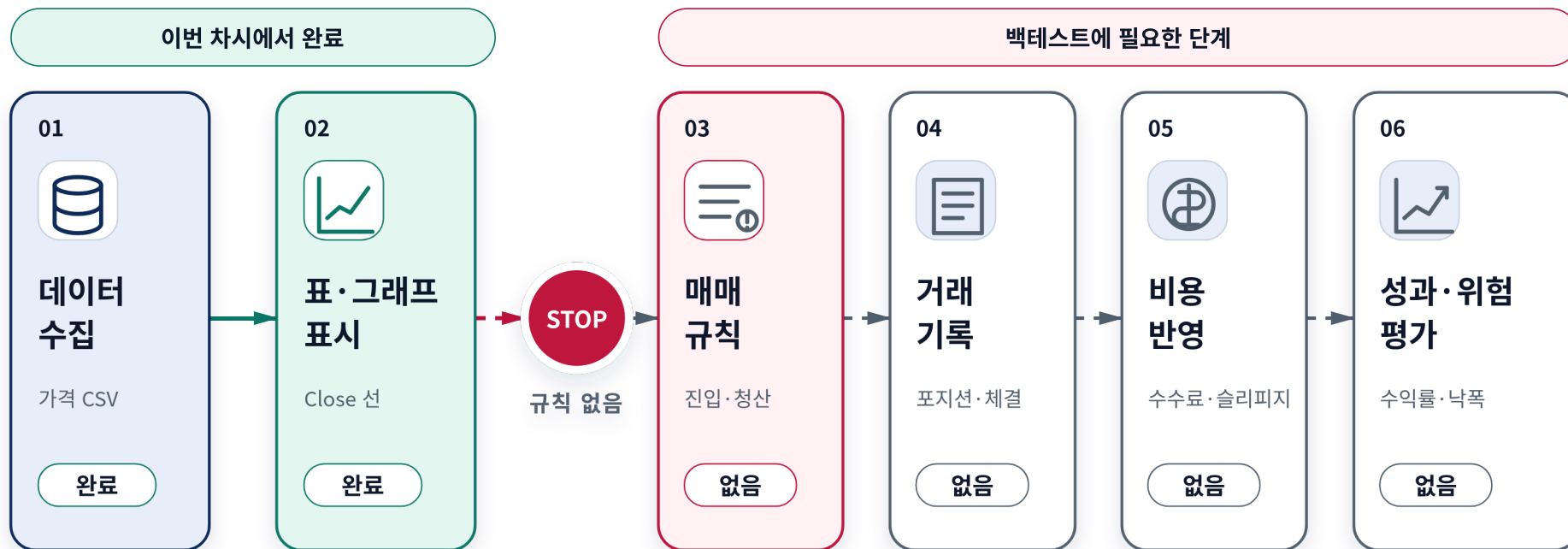


그래프

같은 종가 선이 다시 나타난다

그래프가 나왔어도 아직 백테스트는 아니다

이번 실습은 데이터 수집·저장·표시에서 멈춥니다.



이번에 한 일과 아직 하지 않은 일을 구분한다

결과를 설명할 때 확인한 범위를 넘어서 말하지 않습니다.

DONE

이번에 확인한 것

- ✓ KODEX 200 가격 CSV
- ✓ 날짜별 종가 그래프
- ✓ 새 커널 전체 실행
- ✓ 종목·기간·필수 열

NOT YET

아직 하지 않은 것

- 매수·매도 규칙
- 포지션과 거래 기록
- 수수료·슬리피지·세금
- 수익률·위험 평가

과정 1 · 영상 4

파일을 직접 열고 처음부터 다시 실행하면 된다

CHECK 01

CSV와 노트북을 직접 확인했다



CHECK 02

새 커널에서 Run All을 마쳤다



CHECK 03

아직 백테스트가 아님을 구분했다



실습 파일은 상세 리포트 맨 아래에서 내려받을 수 있습니다.

수고하셨습니다